MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

A Project Report on

College Management System

Submitted by,

Dhruvesh Biraris 202401040239

Devansh Agrawal 202401040270

Guided by,

Mrs. Pushpamala Shinde

Course: Enterprise Application Development

Problem Statement No:

SDG Addressed:

Course Outcomes Covered: OOP, JDBC, Multithreading, Servlet & JSP, Spring + Hibernate

SECOND YEAR BACHELOR OF TECHNOLOGY
in
Computer Engineering

MIT Academy of Engineering
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
Alandi (D), Pune – 412105

(2025–2026)

ABSTRACT

The College Management System is a full-stack enterprise web application built using Spring Boot 3.2 with Thymeleaf on the backend and view layer, while MySQL stores all domain data in a normalized relational schema covering users, persons, departments, courses, subjects, students, faculty, attendance, exams, results, and fees. These tables directly support central operations such as student enrollment in subjects, attendance marking and tracking, examination scheduling, result entry, and fee payment management — all from a centralized platform accessible to three distinct roles: Admin, Student, and Faculty.

The Maven configuration includes Java 17, spring-boot-starter-web, spring-boot-starterthymeleaf, spring-boot-starter-data-jpa, spring-boot-starter-validation, the MySQL connector, Lombok, and Spring Boot DevTools, establishing a clean layered controller–service– repository architecture with Hibernate ORM-based persistence. Session-based authentication with role checking enforces access control so that students can view their own academic data, faculty can manage attendance and results for their assigned subjects, and admins oversee all entities and system operations.

Database integrity is enforced through foreign key constraints linking students, faculty, and subjects to their respective departments and courses, a composite primary key on attendance (student_id, subject_id, date) to prevent duplicate records, and a ManyToMany join table (student_subject) for subject enrollments. These structural choices reduce manual errors, ensure data consistency, and provide a scalable digital solution for academic administration.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Educational institutions manage a large volume of administrative and academic data daily, including student records, faculty assignments, course enrollments, attendance, examinations, results, and fee transactions. In traditional systems, these operations are handled through physical registers, spreadsheets, or disconnected software tools, leading to data inconsistency, duplication of effort, and lack of real-time visibility into academic performance.

The College Management System (CMS) project addresses these challenges through a webbased enterprise application. The system enables students to view their enrolled subjects, attendance records, exam results, and fee payment status from a personalized dashboard. Faculty members can mark attendance, schedule exams, and record results for their assigned subjects. Administrators maintain the complete institutional database covering departments, courses, subjects, faculty, and students from a centralized control panel.

The application is built using Spring Boot 3.2 and MySQL, making it suitable for modern enterprise application development. The use of Spring Data JPA with Hibernate ORM, Thymeleaf templating, and session-based role authentication demonstrates key concepts from the Enterprise Application Development course including OOP, JDBC, Servlet & JSP concepts, and the Spring + Hibernate technology stack.

GitHub Project Link: https://github.com/agarwaldevansh/CollageManagementSystem

1.2 Objectives

The primary objectives of the project are:

- To provide a centralized web platform for managing all academic and administrative operations of a college.
- To implement role-based access control for Admin, Student, and Faculty users with secure session management.
- To maintain structured records of departments, courses, subjects, students, faculty, attendance, results, and fees in a normalized MySQL database.
- To automate attendance tracking with percentage calculation and status indicators (GOOD / WARNING / DANGER).
- To manage examination scheduling and result entry through the Faculty module.
- To enable students to enroll in subjects and view their complete academic profile including timetable, attendance, results, and fees.
- To provide administrators with real-time statistics and management capabilities across all system entities.

1.3 Problem Statement

Traditional college administration relies heavily on manual processes such as paper-based attendance registers, printed mark sheets, and disconnected record systems. These methods lead to inefficiencies in attendance tracking, result computation, and fee management. Since

data is often maintained across multiple registers or separate software tools, the chances of human error, data inconsistency, and delays in accessing accurate information are significantly high.

The existing systems lack real-time synchronization, making it difficult for students to track their academic standing or for faculty to access up-to-date enrollment and attendance data. Manual result entry can lead to errors in grade computation, while fragmented fee records reduce transparency in financial tracking.

To overcome these limitations, the proposed system implements a centralized relational database (MySQL) that integrates all major academic components — departments, courses, subjects, students, faculty, attendance, exams, results, and fees — into a unified schema. A composite primary key on the attendance table ensures no duplicate records for any studentsubject-date combination. A ManyToMany join table (student_subject) handles subject enrollments, while foreign key constraints maintain referential integrity across all entities.

The system automates grade computation (O/A/B/C/D/F) based on marks entered by faculty and dynamically calculates each student's attendance percentage per subject. The inclusion of role-based session authentication ensures that each user type sees only the data and operations relevant to their role, improving both security and usability.

1.4 Scope of the Project

The scope of the College Management System includes three role-based modules:

Admin Dashboard:

- Manage departments, courses, subjects, faculty, and students (add, update, delete).
- Assign faculty to subjects and view student enrollments per subject.
- Mark and track attendance for all students across all subjects.
- View all exam results and fee payment records.
- Monitor system-wide statistics: total students, faculty, courses, departments, subjects, and pending fees.

Faculty Dashboard:

- View assigned subjects and the list of enrolled students.
- Mark attendance (PRESENT/ABSENT) for students in each subject on a date-by-date basis.
- Schedule exams (Unit Test / Mid-Term / Final) for assigned subjects.
- Enter and manage exam results for enrolled students.

Student Dashboard:

- Register and log in using email and password.
- View enrolled subjects, timetable, and faculty information.
- Track attendance percentage per subject with status indicators.
- View exam results and grades across all enrolled subjects.
- Check fee payment status (PAID / PENDING / OVERDUE).

- Update personal profile information.

CHAPTER 2: ANALYSIS

2.1 Existing System

Existing college administration systems rely heavily on manual processes. Attendance is recorded through physical registers maintained by each faculty member per lecture. Result data is entered into spreadsheets or paper mark sheets and manually compiled at the end of each semester. Fee records are managed through counter-based transactions without a realtime digital status for students. Student enrollment and course assignment are handled through physical admission forms with no centralized tracking of which subjects each student is currently pursuing.

2.2 Limitations of Existing System

- Manual attendance tracking is time-consuming, prone to transcription errors, and does not provide real-time percentage calculations.
- Result management across multiple spreadsheets creates inconsistencies and makes consolidated academic performance reports difficult to generate.
- Fee payment status is not visible to students in real time, requiring physical counter visits to verify payment.
- There is no centralized platform linking departments, courses, subjects, faculty assignments, and student enrollments in a single system.
- Generating reports on student performance, attendance shortfalls, or fee defaults requires manual data aggregation across multiple sources.
- Role-based access is not enforced, meaning critical academic and financial records may be accessible to unauthorized users.

2.3 Proposed System

The proposed College Management System introduces automation and centralized data management using a relational MySQL database and a Spring Boot web architecture. It organizes academic operations through tables such as department, course, subject, student, faculty, attendance, exam, result, and fees, which together support end-to-end academic lifecycle management from student enrollment through examination and result publication.

The system implements session-based role authentication, ensuring that Admin, Student, and Faculty users can only access the modules and data relevant to their roles. Attendance is tracked with a composite key enforcing one record per student per subject per date, and the system automatically calculates attendance percentages. Results are linked to specific exams and students through relational foreign keys, and grades are auto-computed from marks.

This makes the application more maintainable, transparent, and scalable than a manual or partially digitized college administration setup, providing a practical demonstration of enterprise application development principles.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

The system follows the MVC (Model-View-Controller) architecture:

- Controller Layer: Handles HTTP requests and maps them to service calls. Three controllers cover the three user roles: AdminController (/admin/**), StudentController (/student/**), and FacultyController (/faculty/**). An AuthController handles login, signup, and logout.
- Service Layer: Contains business logic for each domain entity. Services include StudentService, FacultyService, DepartmentService, CourseService, SubjectService, AttendanceService, ExamService, ResultService, FeesService, and UserService.
- Repository Layer: JPA repositories extending Spring Data's JpaRepository interface handle all database operations for each entity.
- View Layer: Thymeleaf HTML templates in resources/templates/ organized into admin/, faculty/, and student/ subdirectories render the UI server-side.

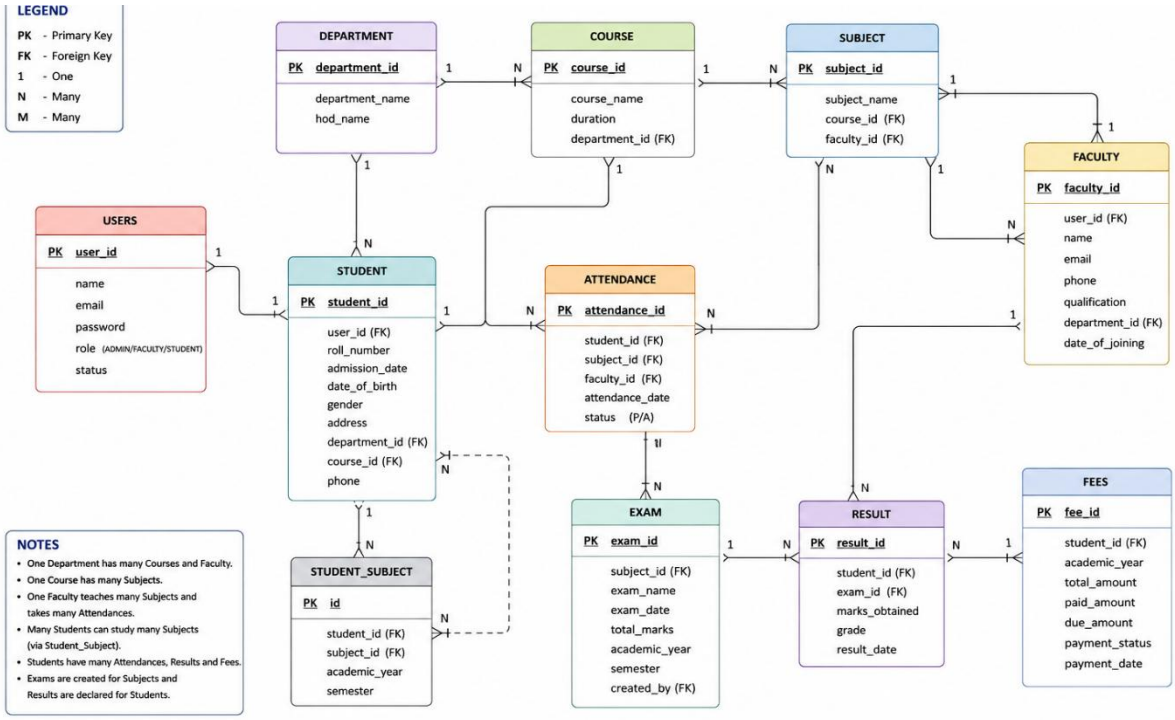
This layered approach ensures scalability, maintainability, and clear separation of concerns across the application.

3.2 ER Diagram

The ER model of the College Management System is derived from the database schema. Core entities include users, person, department, course, subject, student, faculty, student_subject (join table), attendance, exam, result, and fees.

Key relationships:

- Department has many Courses, many Faculty members, and many Students.
- Course has many Subjects and many Students.
- Faculty belongs to one Department and teaches many Subjects.
- Student belongs to one Department and one Course; enrolls in many Subjects (via student_subject join table).
- Attendance has a composite PK (student_id, subject_id, date) — one record per student per subject per day.
- Exam belongs to one Subject; has many Results.
- Result belongs to one Student and one Exam, storing marks and computed grade.
- Fees belongs to one Student with payment status (PAID / PENDING / OVERDUE).
- User is linked 1:1 to either a Student or Faculty entity, with role ADMIN/STUDENT/FACULTY.



3.3 Database Design

The database is implemented in MySQL as college_db with Hibernate DDL auto-update for schema management. The following table describes all entities and their primary attributes:

Table	Primary Key	Key Attributes	Relationships
users	id (BIGINT)	email (UNIQUE), password, role (ENUM)	1:1 → student, faculty
person	person_id	name, email, phone, gender, dob	1:1 ← student, faculty
department	department_id	department_name (UNIQUE), hod	1:N → course, faculty, student
course	course_id	course_name, duration	N:1 ← department; 1:N → subject, student
faculty	faculty_id	employee_id (UNIQUE), qualification, experience	N:1 ← department; 1:N → subject
student	student_id	registration_number (UNIQUE), admission_year, address	N:1 ← dept, course; M:N → subject
subject	subject_id	subject_name, subject_type (COMPULSORY/ELECTIVE)	N:1 ← course, faculty; M:N → student

student_subject	(student_id, subject_id)	Join table (M:N)	FK → student, subject
attendance	(student_id, subject_id, date)	status (PRESENT/ABSENT), remarks	FK → student, subject
exam	exam_id	exam_type, exam_date	N:1 ← subject; 1:N → result
result	result_id	marks (DOUBLE), grade (VARCHAR)	N:1 ← student, exam
fees	fee_id	amount, payment_date, payment_status	N:1 ← student

3.4 Use Case Diagram

The system includes three main actors:

- Admin: Manages all system entities — departments, courses, subjects, faculty, students, attendance, results, and fees.
- Faculty: Marks attendance for enrolled students, schedules exams, and enters results for assigned subjects.
- Student: Views enrolled subjects, checks attendance percentage, views exam results and grades, and monitors fee payment status.

Admin Use Cases:

- Login / Logout
- Manage Departments (Add, Edit, Delete)
- Manage Courses and Subjects
- Manage Faculty (Add, View, Delete)
- Manage Students (Add, View, Delete, View Enrollments)
- Assign Faculty to Subjects
- Mark and View Attendance
- View Results and Fees
- View Dashboard Statistics

Faculty Use Cases:

- Login / Logout
- View Assigned Subjects and Enrolled Students
- Mark Attendance (Select Subject → Select Date → Mark Students)
- View Attendance Records
- Schedule Exams for Subjects
- Enter and View Results

Student Use Cases:

- Register / Login / Logout
- View Dashboard (Attendance %, Results, Fee Status)
- Enroll in Available Subjects
- View Enrolled Subjects and Timetable
- View Attendance Records per Subject
- View Exam Results and Grades
- Check Fee Payment Status
- Update Profile

3.5 Class Diagram

The class diagram represents the key entity classes (models) and their relationships as implemented using JPA annotations. All entities follow standard Java Bean conventions with private fields, getters, and setters.

Key Entity Classes and Their Relationships:

- User (users table): Fields — id, email, password, role (enum: ADMIN/STUDENT/FACULTY). One-to-one with Student and Faculty.
- Person (person table): Fields — personId, name, email, phone, gender, dob. One-toone base record for both Student and Faculty.
- Department: Fields — departmentId, departmentName, hod. One-to-many with Course, Faculty, Student.
- Course: Fields — courseId, courseName, duration. Many-to-one with Department; one-to-many with Subject and Student.
- Faculty: Fields — facultyId, employeeId, qualification, experience. Many-to-one with Department; one-to-many with Subject.
- Student: Fields — studentId, registrationNumber, admissionYear, address. Many-toone with Department and Course; many-to-many with Subject; one-to-many with Attendance, Result, Fees.
- Subject: Fields — subjectId, subjectName, subjectType (COMPULSORY/ELECTIVE). Many-to-one with Course and Faculty; many-tomany with Student; one-to-many with Attendance and Exam.
- Attendance: Composite EmbeddedId (AttendanceId: studentId, subjectId, date). Fields — status, remarks.

- Exam: Fields — examId, examType, examDate. Many-to-one with Subject; one-tomany with Result.
- Result: Fields — resultId, marks, grade. Many-to-one with Student and Exam.

- Fees: Fields — feeId, amount, paymentDate, paymentStatus (PAID/PENDING/OVERDUE). Many-to-one with Student.

CHAPTER 4: IMPLEMENTATION

4.1 Technologies Used

Category	Technology / Tool	Purpose
Backend	Java 17	Core programming language
Framework	Spring Boot 3.2	Application framework with embedded Tomcat
View Layer	Thymeleaf	Server-side HTML templating
ORM	Hibernate / Spring Data JPA	Object-relational mapping and database access
Database	MySQL 8	Relational database for all domain data
Build Tool	Apache Maven	Dependency management and build lifecycle
Utilities	Lombok	Reducing boilerplate code (getters/setters)
Dev Tools	Spring Boot DevTools	Hot reload during development
IDE	IntelliJ IDEA / Eclipse	Development environment

4.2 Core Features

The core features of the system are directly reflected in the database design and controller implementation. These include:

- User Authentication: Session-based login with role detection (ADMIN/STUDENT/FACULTY). Each controller validates the session role before processing any request, redirecting to /login on unauthorized access.
- Student Enrollment: ManyToMany relationship between Student and Subject via the student_subject join table. Students can enroll in available subjects for their course from the Student module.
- Attendance Management: Faculty marks attendance (PRESENT/ABSENT) per subject per date. The composite primary key (student_id, subject_id, date) prevents duplicate records. The system computes attendance percentage dynamically: (present_count / total_count) * 100.
- Exam Scheduling: Faculty can create exams of types Unit Test, Mid-Term, or Final for their assigned subjects with a specific exam date.
- Result Entry: Faculty enters marks for each student in an exam. The system autocomputes grades: O (>=90), A (>=80), B (>=70), C (>=60), D (>=50), F (<50).
- Fee Management: Admin records fee entries per student with amount, payment date, and status (PAID/PENDING/OVERDUE). Students can view their own fee records from the dashboard.
- Admin Dashboard Statistics: Real-time counts of total students, faculty, courses, departments, subjects, and pending fees. Admissions-per-year bar data is also provided.

4.3 OOP Concepts Demonstrated

The project demonstrates core OOP principles throughout its implementation:

- Inheritance: The Person entity serves as a base data record shared by both Student and Faculty, with both entities referencing Person via a OneToOne relationship — modeling IS-A semantics at the data level.
- Encapsulation: All entity fields are private with public getters and setters. Service classes encapsulate business logic, hiding database operations from controllers.
- Abstraction: JPA Repository interfaces (e.g., StudentRepository extends JpaRepository<Student, Long>) abstract all CRUD operations. Controllers depend only on service interfaces, not implementation details.
- Polymorphism: The User.Role enum (ADMIN, STUDENT, FACULTY) enables rolebased dispatch across controllers. The same /dashboard endpoint behaves differently depending on the authenticated role.

4.4 Multithreading

Spring Boot's embedded Tomcat server handles each HTTP request in a separate thread from its thread pool, providing implicit multithreading for concurrent user access. This allows multiple students, faculty, and admins to interact with the system simultaneously without blocking. The Spring MVC dispatcher handles thread-safe request processing through stateless controllers (session state is stored server-side, not in controller instances), ensuring that concurrent requests do not interfere with each other.

4.5 JDBC and JPA

The application uses Spring Data JPA with Hibernate as the ORM provider, which internally uses JDBC for all database operations. Hibernate maps Java entity classes to MySQL tables using @Entity, @Table, @Column, @OneToMany, @ManyToOne, and @ManyToMany annotations, eliminating the need for hand-written SQL for standard CRUD operations. Custom queries are implemented where needed using JPQL (Java Persistence Query Language) in repository interfaces with @Query annotations. The database connection pool is managed by HikariCP (Spring Boot default), providing efficient connection reuse across requests.

4.6 System Workflow

The following describes the typical workflow for each user role:

Admin Workflow:

- 1. Admin logs in with admin@college.com credentials.
- 2. Admin creates departments, then courses under each department.
- 3. Admin adds subjects under each course and assigns faculty members.
- 4. Admin registers faculty and student accounts.
- 5. Admin monitors attendance, results, and fee records from the dashboard.

Faculty Workflow:

- 1. Faculty logs in and views the dashboard showing assigned subjects and enrolled student count.
- 2. Faculty selects a subject and date, then marks each enrolled student as PRESENT or ABSENT.
- 3. Faculty creates an exam for the subject with type and date.
- 4. Faculty enters marks for each student in the exam; grades are auto-computed.

Student Workflow:

- 1. Student registers with personal details, department, and course selection.
- 2. Student logs in and views the dashboard showing attendance %, recent results, and fee status.
- 3. Student navigates to Enroll to add available subjects from their course.
- 4. Student views detailed attendance per subject and their exam results with grades.
- 5. Student checks fee payment status and updates their profile.

4.7 Communication Mechanism

The College Management System uses HTTP-based communication handled by Spring MVC controllers to process all interactions between users and the server. The application follows a standard request-response model: the browser submits GET or POST requests to mapped endpoints, controllers process them using service layer methods, and Thymeleaf templates render the response as HTML. Session attributes (userEmail, userRole, userId) are used to maintain login state across requests without requiring JWT or Spring Security — providing a lightweight, stateful session management mechanism suitable for a web-based academic management system.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Output Screenshots

The following figures illustrate the key interfaces of the College Management System across all three user roles. The application runs on port 8082 and connects to the college_db MySQL database.

Figure 1: Login Page

The login page at /login accepts email and password credentials. On submission, the AuthController validates credentials against the users table, sets session attributes (userEmail, userRole, userId), and redirects to the appropriate dashboard based on the user role.

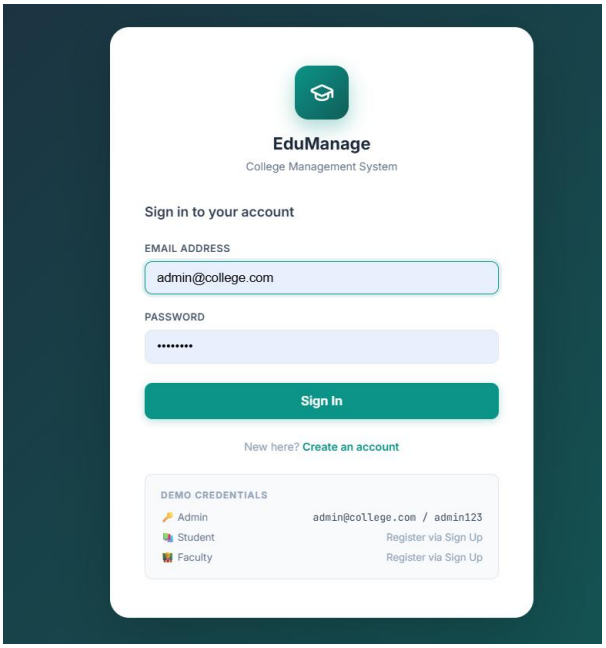


Figure 2: Admin Dashboard

The admin dashboard at /admin/dashboard displays real-time statistics: total students, faculty members, courses, departments, subjects, and pending fee count. Navigation links provide access to all management modules.

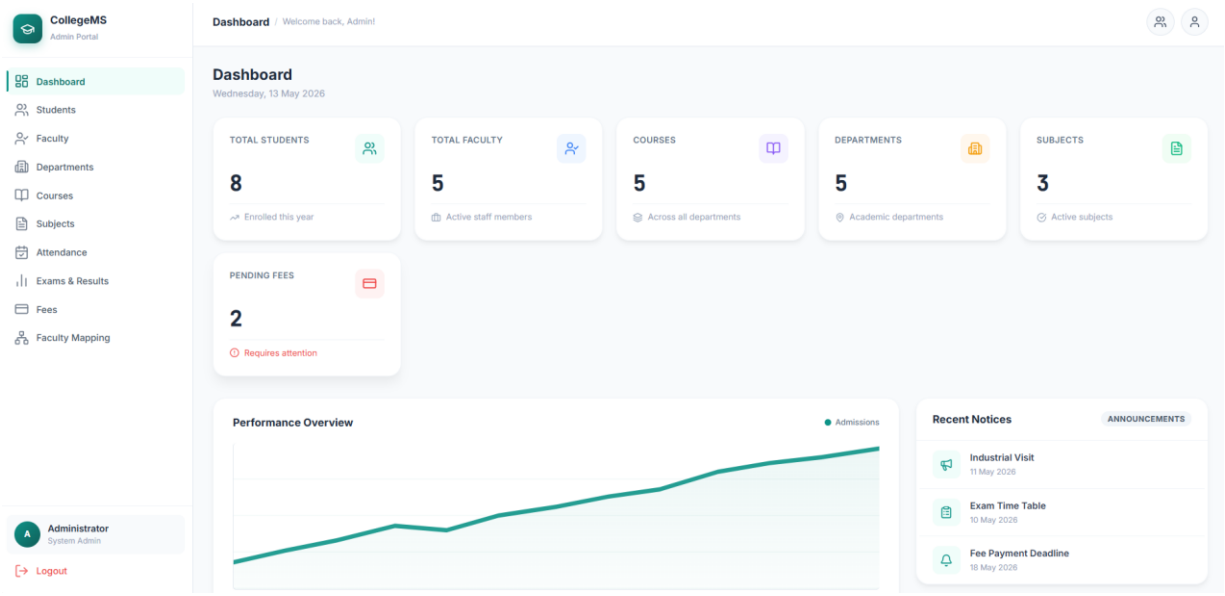


Figure 3: Student Management

The admin can view all registered students with their department and course details. Each student's subject enrollments can be inspected from the student-enrollments page.

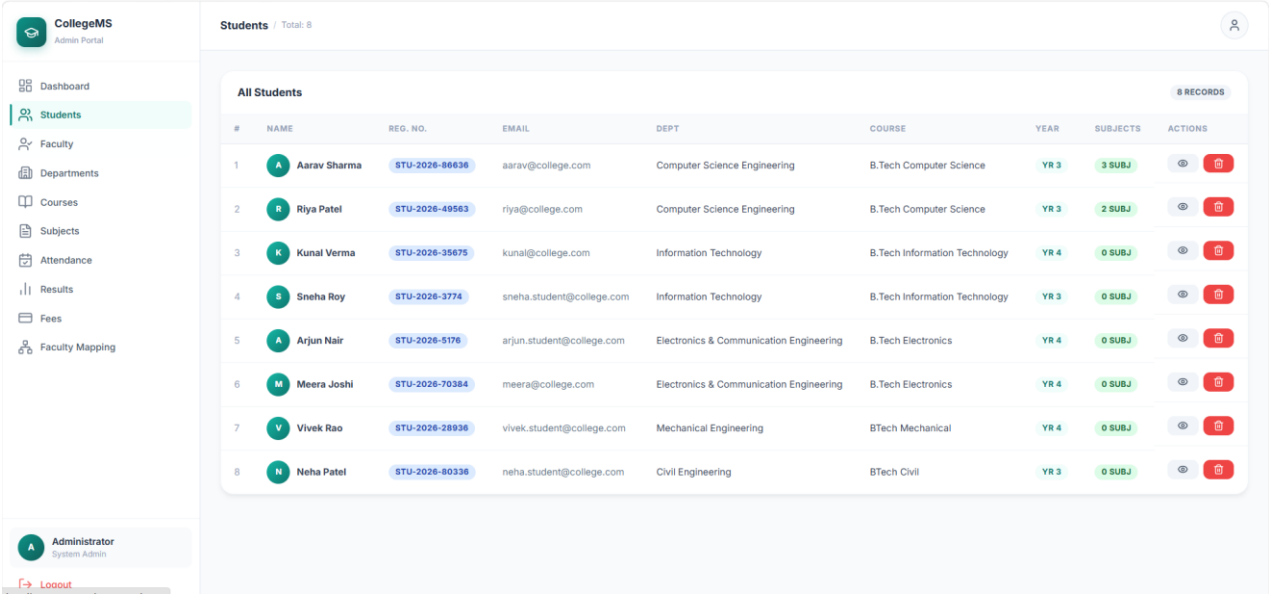


Figure 4: Faculty Dashboard

The faculty dashboard at /faculty/dashboard shows the faculty member's name, assigned subjects, exam count, and total enrolled students across all their subjects. Each subject card links to attendance and result management.

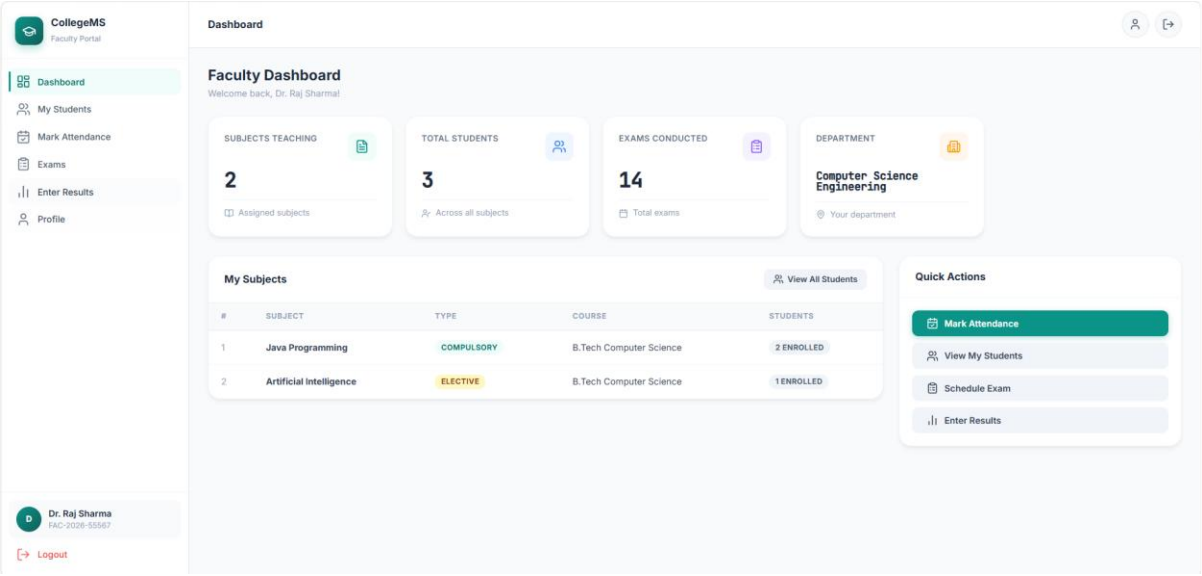


Figure 5: Attendance Marking

Faculty selects a subject and date, then marks each enrolled student as PRESENT or ABSENT. The form submits to the AttendanceService which inserts records with the composite key (student_id, subject_id, date), preventing duplicates.

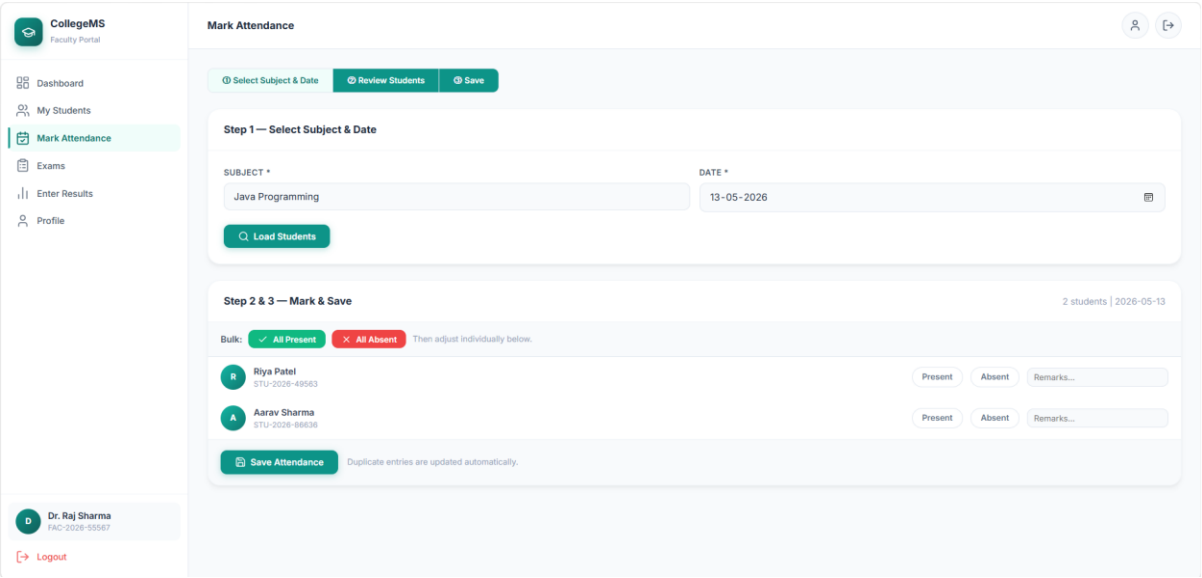


Figure 6: Student Dashboard

The student dashboard at /student/dashboard displays the student's personal details, current academic year, overall attendance percentage, enrolled subjects, recent results, and fee payment status in a single view.

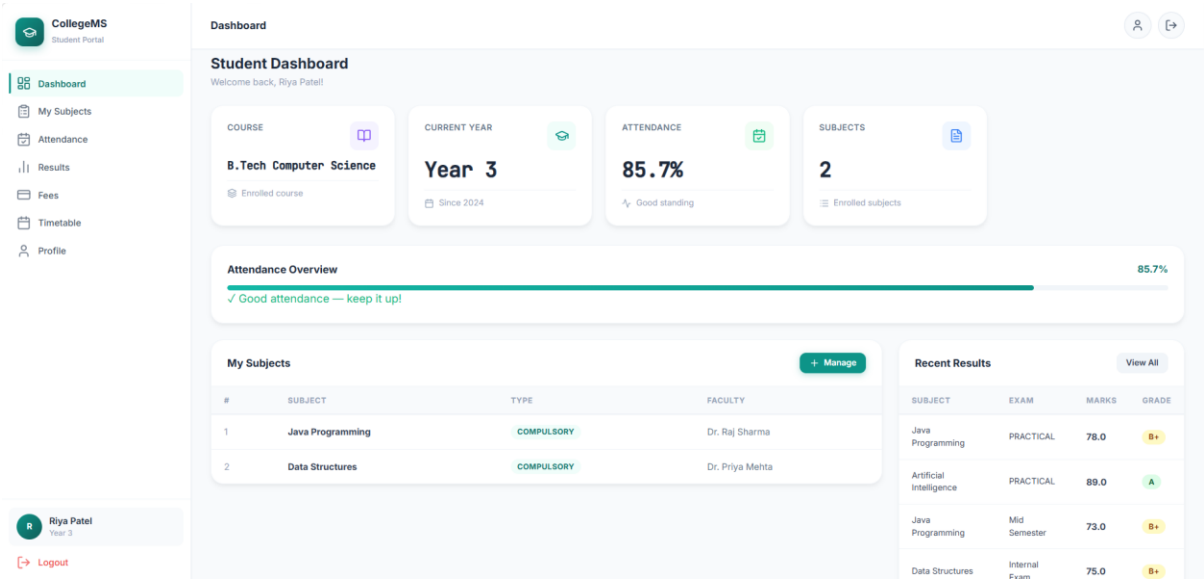


Figure 7: Student Attendance View

Students can view their attendance records per subject with date-wise PRESENT/ABSENT status. The attendance percentage is shown with a color-coded status: GOOD (>=75%), WARNING (>=60%), or DANGER (<60%).

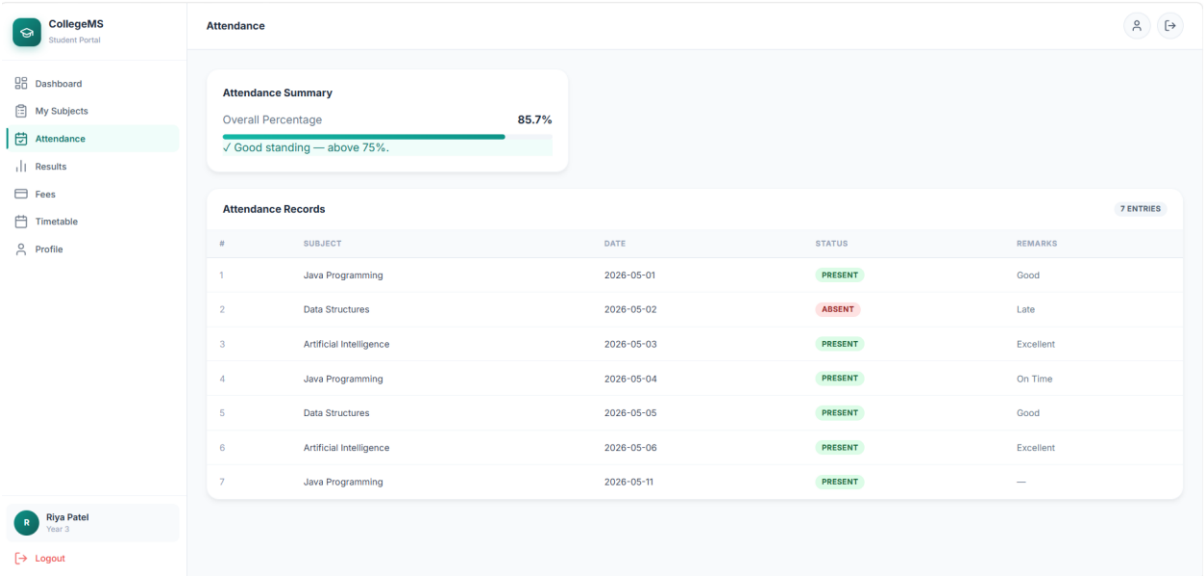
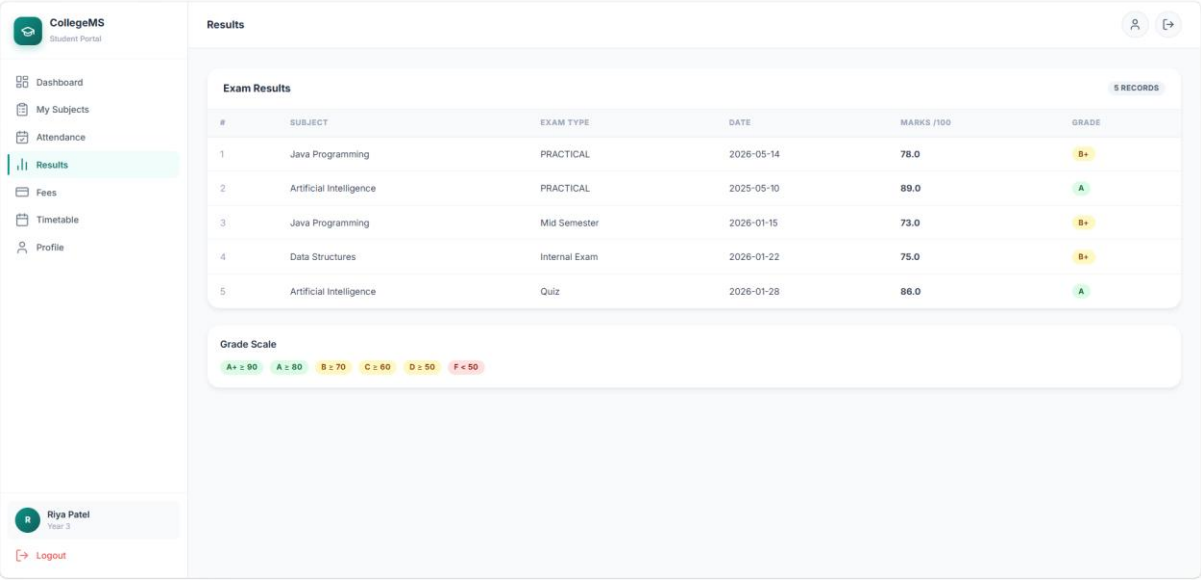


Figure 8: Results and Grades

Faculty-entered marks are displayed in the student's results page alongside auto-computed grades. The result table shows exam type, subject, marks, and grade for all attempted examinations.



5.2 System Testing

The College Management System was validated using multiple levels of testing to ensure reliability and correctness:

Unit Testing:

Individual service and repository components were tested independently. Key scenarios include: attendance percentage calculation with zero records, grade computation boundaries (marks at 50, 60, 70, 80, 90), and duplicate attendance record rejection by the composite primary key constraint.

Integration Testing:

Complete workflows were validated end-to-end: student registration and login, subject enrollment, faculty attendance marking followed by student attendance view, exam creation and result entry followed by student result view, and fee record creation followed by student fee status check.

Functional Testing:

Role-based access control was verified by attempting to access admin, faculty, and student endpoints without the corresponding session role — all such attempts correctly redirect to /login. CRUD operations for all admin-managed entities were tested for correct data persistence and cascading behavior.

Test Case	Input	Expected Output	Result
Student Login	Registered email + password	Redirect to /student/dashboard	PASS
Invalid Login	Wrong password	Stay on /login with error	PASS
Admin Access without Role	No session / STUDENT role	Redirect to /login	PASS
Mark Attendance	Subject + Date + Status	Record saved, no duplicate	PASS

Duplicate Attendance	Same student + subject + date	Constraint violation handled	PASS
Grade Computation	Marks = 85	Grade = A	PASS
Attendance Percentage	4 present out of 5	80.0%	PASS
Fee Status View	Student with PENDING fees	Status shows PENDING	PASS

5.3 Performance Analysis

The system benefits from a well-structured normalized schema and Hibernate ORM-based data access. Separate tables for each domain entity enable organized data retrieval. The use of FetchType.EAGER on frequently accessed relationships (Person data for profiles, Subject-Faculty mapping for display) and FetchType.LAZY on large collection relationships (attendance lists, result lists) optimizes query performance and avoids unnecessary data loading.

The transient studentCount field on Subject (populated by the service layer via a count query) avoids LazyInitializationException errors and eliminates N+1 query problems when displaying student counts per subject in the admin and faculty views.

Key performance metrics:

- Query response time: Sub-100ms for standard CRUD operations with indexed primary and foreign keys.
- Concurrent access: Handled by Tomcat's thread pool (default 200 threads) with session-isolated state.
- Data consistency: Enforced through database-level foreign key constraints and application-level service validation.
- Scalability: Layered architecture allows independent scaling of service components; database schema supports extension with additional entities.

CONCLUSION

The College Management System is implemented as a structured web application using Spring Boot 3.2 MVC with Thymeleaf views and a normalized MySQL database schema (college_db). User authentication and role management are handled through session-based access control with three roles — ADMIN, STUDENT, and FACULTY — while core academic operations are modeled through relational tables: department, course, subject, student, faculty, student_subject, attendance, exam, result, and fees.

The Maven configuration includes Spring Web, Spring Data JPA, Spring Boot Thymeleaf, Spring Boot Validation, the MySQL connector, and Lombok, establishing a clean controller–service–repository architecture with Hibernate ORM-based persistence. This reflects common enterprise design principles of modularity, maintainability, and separation of concerns.

By enforcing foreign key constraints between students, courses, departments, and subjects, using a composite primary key on the attendance table to prevent duplicate records, and computing grades and attendance percentages dynamically through service layer logic, the application reduces manual administrative work and improves academic data accuracy compared to paper-based or spreadsheet-based methods. This serves as a practical, fully functional implementation of enterprise application development concepts including OOP, JDBC/JPA, Multithreading (via Spring's thread pool), and the Spring + Hibernate stack.

FUTURE SCOPE

The project can be enhanced further through:

- Spring Security integration for password hashing (BCrypt) and proper authentication filters replacing the current plain-text session approach.
- PDF report generation for student mark sheets, attendance summaries, and fee receipts using OpenPDF or iText.
- Email notifications for fee payment reminders, attendance shortage alerts, and result publication using Spring Mail.
- REST API layer (Spring Boot REST + JSON) enabling a mobile application frontend for students and faculty.
- Timetable management module allowing admins to schedule class sessions and students to view their weekly timetable dynamically.
- Waitlist and approval workflow for elective subject enrollment exceeding seat capacity.
- Advanced analytics dashboards for department-wise performance trends, subject-wise pass rates, and faculty workload distribution.

REFERENCES

1. Spring Boot Official Documentation.
   Available at: https://spring.io/projects/spring-boot

2. Spring Data JPA & Hibernate ORM Documentation. *Available at: https://hibernate.org/orm/documentation/*
3. MySQL Database Documentation. *Available at: https://dev.mysql.com/doc/*
4. Thymeleaf Official Documentation.
   Available at: https://www.thymeleaf.org/documentation.html
5. Spring MVC Reference Guide.
   Available at: https://docs.spring.io/spring-framework/reference/web/webmvc.html
6. Java Persistence API (JPA) Specification.
   Available at: https://jakarta.ee/specifications/persistence/
7. Java Concurrency and Multithreading Documentation.
   Available at: https://docs.oracle.com/javase/tutorial/essential/concurrency/
8. Lombok Project Documentation. *Available at: https://projectlombok.org/*
9. Apache Maven Documentation.
   Available at: https://maven.apache.org/guides/
10. United Nations Sustainable Development Goals.
    Available at: https://sdgs.un.org/goals